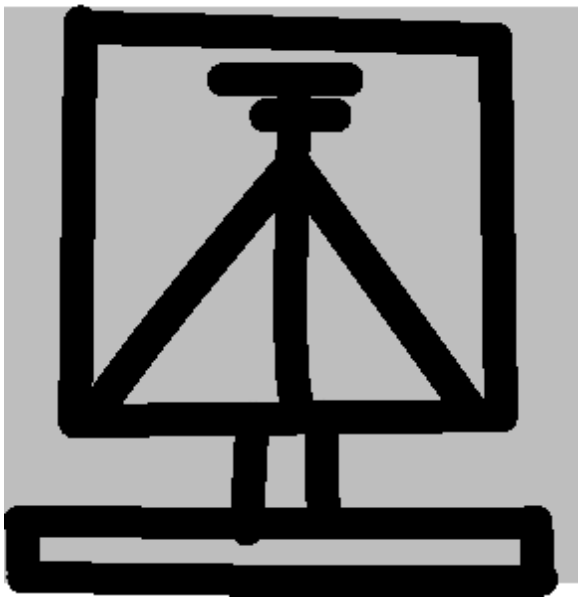
# Module B4 "Géomatique + TI"

# B4a "INTERLIS 2"



# Table des matières


1. Nouveautés d'INTERLIS - Vue d'ensemble 5
1.1. Changements au niveau de la syntaxe 6
1.2. Vue d'ensemble 7
1.3. Aperçu de la structure complète 8
1.4. Utiliser des définitions d'un autre modèle 9
1.5. Format d'échange 10
1.6. Dépendances entre conteneurs 10
1.7. Livraison incrémentielle des données 11
2. Unités et systèmes de références 12
2.1. Unités de base 12
2.2. Unités dérivées 12
2.3. Unités composées 13
2.4. Unités structurées 13
2.5. Unités prédéfinies par INTERLIS 13
2.6. Utilisation des définitions d'unités d'un autre modèle 14
2.7. Coordonnées 14
2.8. Coordonnées et systèmes de référence 15
2.9. Coordonnées et indication de la rotation 15
3. Relations, rôle, intensité, cardinalité 16
3.1. Définitions 16
3.1.1 Relation : 16
3.1.2 Rôle : 16
3.1.3 Représentation graphique 16
3.2. Syntaxe INTERLIS 2 17
3.2.1 Aperçu 17
3.2.2 Association en général 17
3.2.3 Association simple 17
3.2.4 Cardinalité 18
3.2.5 Intensité de la relation 18
3.2.6 Relations ordonnées 19




*© CF-geo, Mattei Alain 09.2019*

**4. Condition de cohérence, expression logique, fonction 20**
**4.1. Exemple : INTERLIS1 → INTERLIS 2 20**
**4.2. Définitions et aperçu 20**
4.2.1 Définitions 20
**4.3. Détails des conditions de cohérences 21**
4.3.1 Condition d'unicité 21
4.3.2 Condition obligatoire 21
4.3.3 Exigence d'existence 22
**4.4. Fonctions 22**
**5. Héritage pour éléments de base 24**
**5.1. L'héritage 24**
5.1.1 Héritage et classes abstraites 24
5.1.2 Le multihéritage 24
**5.2. Le polymorphisme, conséquence de l'héritage ! 24**
5.2.1 Lecture polymorphique avec INTERLIS 2 25
5.2.2 Compatibilité des modèles, autour d'un noyau commun en utilisant l'héritage 25
**5.3. Etendre un thème 25**
5.3.1 Classe simple 26
5.3.2 Spécialisation d'une classe et changer son nom 26
5.3.3 Spécialisation d'une classe sans changer son nom 26
5.3.4 Forcer l'héritage 27
5.3.5 Empêcher l'héritage 27
5.3.6 Attribut avec un domaine défini incomplet 27
5.3.7 Attribut avec un domaine restreint 28
5.3.8 Attribut avec un intervalle qui ne pourra plus être limité 28
5.3.9 Spécialisation de la définition du type d'attribut (domaine) 28
5.3.10 Spécialisation d'une énumération (en ajoutant de nouvelles valeurs) 29
5.3.11 Spécialisation d'une énumération (en ajoutant une contrainte) 29
5.3.12 Exemple pratique 30
5.3.13 Associations 30

6. Identifiants d'objets 31
7. Outils INTERLIS 2 32
7.1. Compilateur INTERLIS ili2c 32
7.1. Checker ilivalidator 32
7.2. Editeur UML/INTERLIS 32
7.3. Documents et outils pour INTERLIS 2 33
7.4. Outil pour valider des données en INTERLIS 2 33
7.5. Transformation d'un modèle INTERLIS 1 en 2 33
7.6. Transformation des données INTERLIS 1 en 2 33
7.7. Outils XML 33
7.8. Liens utiles 33
8. Bibliographie 34
9. Exercices – INTERLIS 2 35
9.1. Transcription 1:1 du modèle zone1.ili en INTERLIS 2 35
9.2. Modélisation des unités 35
9.3. Modélisation des associations 36
9.4. Modélisation avec éditeur UML/INTERLIS 37
9.5. Exemples d'utilisation de INTERLIS 2 37


*© CF-geo, Mattei Alain 09.2019*

# 1. Nouveautés d'INTERLIS - Vue d'ensemble

**INTERLIS** = **INTER L**and-**I**nformations-**S**ystemen (entre les SIT en français).

Avec l'arrivée d'INTERLIS 2, le langage de description existant INTERLIS 1 est un peu modifié et surtout largement étendu. Alors qu'INTERLIS 1 permettait la description d'un **modèle relationnel**, INTERLIS 2 permet la description d'un **modèle orienté objet**. De ce fait, les termes changent et des nouveaux concepts tels que l'héritage apparaissent. Le format de transfert utilise dorénavant l'encodage **XML** (eXtensible Markup Language).

Historique récent de INTERLIS 2 :

**2006 INTERLIS 2.3**
- Norme suisse SN612031
- Version actuellement utilisée

**2017 INTERLIS 2.4**
- Norme **eCH-0031** version 2.0, entièrement compatible avec INTERLIS 2.3
- Grands changements et simplifications par rapport à la version 2.3 pour le **transfert de données** (fichier XML)
- **Définitions du schéma XML** (fichier XSD) intégrées dans la norme
- **Géométries multiples** avec meilleure compatibilité avec OGC
- Suppression des **attributs de lignes**
- Noms de balises d'objets raccourcis au nom de la classe (fichier XML)
- Etc…


*© CF-geo, Mattei Alain 09.2019*

## 1.1. Changements au niveau de la syntaxe

```
TRANSFER PetitExemple;

MODEL ExemplePetit _

  DOMAIN
    CoordExpl = COORD2  623000.00  254000.00
                        626000.00  257000.00;
    OriEcriture = GRADS 0.0 400.0 _;
    Statut  = (valable,projete);

TOPIC Amenagement_du_territoire =

  TABLE ZoneBatie =
   TypZone:
       (ZoneVillage_V1,ZoneHabitation_H1,ZoneHabitation_H2
   ,ZoneUtilitePublique,ZoneSport);
   SurfaceCalculee: _ DIM2 1 999999999;
   Geometrie: AREA WITH (STRAIGHTS,ARCS) VERTEX
       CoordExpl
    WITHOUT OVERLAPS > 0.05;
   ApprobationCommune: _ _ DATE;
   ApprobationCanton: _ _ DATE;
  NO IDENT
  END ZoneBatie;

END Amenagement_du_territoire.
...
```

```
 INTERLIS 2.3;

 MODEL PetitExemple  (fr) AT "http://www.interlis.ch/models"
   VERSION "2005-06-16" =

   UNIT
    Grads = 200.0 / PI [rad];
    SquareMeters [m2] = (m * m);

   DOMAIN
    CoordExpl = COORD 623000.00 .. 626000.00,
                      254000.00 .. 257000.00,
                      ROTATION  2 -> 1;
    OriEcriture = 0.0 .. 400.0 [Grads];
    Statut = (valable, projete);

   TOPIC Amenagement_du_Territoire =

    CLASS ZoneBatie =
       TypZone: MANDATORY  (ZoneVillage_V1,ZoneHabitation_H1
       ,ZoneHabitation_H2,ZoneUtilitePublique,ZoneSport);
       SurfaceCalculee: MANDATORY  1 .. 999999999 [m2];
       Geometrie: MANDATORY  AREA WITH (STRAIGHTS,  ARCS)
        VERTEX CoordExpl WITHOUT  OVERLAPS > 0.05;
       ApprobationCommune:  MANDATORY  INTERLIS_1_DATE;
       ApprobationCanton: MANDATORY  INTERLIS_1_DATE;
    END ZoneBatie;

    END Amenagement_du_Territoire;
  ...
```

```
...
TOPIC Objets_Divers =

  TABLE Batiment =
    Geometrie: _ SURFACE WITH (STRAIGHTS, ARCS) VERTEX CoordExpl
     WITHOUT OVERLAPS > 0.00;
    NoMiseEnquete: _ TEXT*9;
    DatePermis: _ _ DATE;
    Statutinfo: _ Statut;
  IDENT NoMiseEnquete;
  END Batiment;

  TABLE NoPolice =
    Objet: -> Batiment; !! Association 1-mc
    NumeroPol: _ TEXT*4;
    PosNum: _ CoordExpl // Position pres de l'entree du batiment //;
    OriNum: _ OriEcriture;
    HAliNum: OPTIONAL HALIGNMENT; !! Default: Center
    VAliNum: OPTIONAL VALIGNMENT; !! Default: Half
  NO IDENT
  END NoPolice;

  _

END Objets_Divers.

END ExemplePetit.

FORMAT FIX WITH LINESIZE=75, TIDSIZE=10;

CODE
  BLANK = DEFAULT, UNDEFINED = DEFAULT, CONTINUE = DEFAULT;
  TID = I32;
END.
```

```
...
TOPIC Objets_Divers=

  CLASS Batiment =
  Geometrie: MANDATORY SURFACE WITH (STRAIGHTS, ARCS)
     VERTEX CoordExpl WITHOUT OVERLAPS > 0.00;
    NoMiseEnquete: MANDATORY TEXT*9;
    DatePermis: MANDATORY INTERLIS_1_DATE;
    Statutinfo: MANDATORY Statut;
    UNIQUE NoMiseEnquete;
  END Batiment;

  CLASS NoPolice =
    _
    NumeroPol: MANDATORY TEXT*4;
    PosNum: MANDATORY CoordExpl; !! Position pres de l'entree de batiment
    OriNum: MANDATORY OriEcriture;
    HAliNum: _ HALIGNMENT; !! Default: Center
    VAliNum: _ VALIGNMENT; !! Default: Half
  END NoPolice;

  ASSOCIATION AttributNoPolBatiment =
    RoleBatimt -- {1} Batiment;
    RoleNoPol — {0 .. *} NoPolice;
  END AttributNoPolBatiment;

 END Objets_Divers;

END ExemplePetit.
```

## 1.2. Vue d'ensemble

Chap. 2
Chap. 2
Chap. 2
Chap. 2
Chap. 4
Chap. 3

```
INTERLIS 2.3;

MODEL Confederation (fr) AT "http://www.interlis.ch/models"
  VERSION "2005-06-16" =

  IMPORTS CoordSys;

  TOPIC cadastre_des_arbres =
    REFSYSTEM BASKET CoordSysBasket ~CoordSys.CoordSysTopic;

    DOMAIN
    LKoord = COORD 480000.00 .. 850000.00 [INTERLIS.m],
                   60000.00 .. 320000.00 [INTERLIS.m];

    UNIT
    FrancSuisse [CHF] EXTENDS INTERLIS.MONEY;

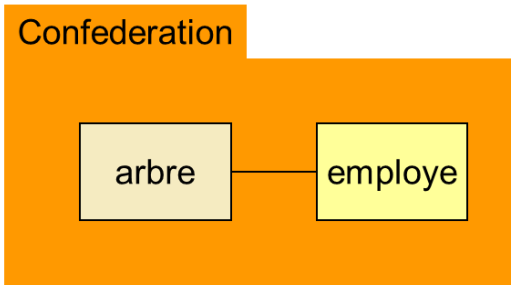
    CLASS arbre =
      position: LKoord;
    END arbre;

    CLASS employe =
      prenom : MANDATORY TEXT*30;
      nom : MANDATORY TEXT*30;
      retraite : BOOLEAN;
      salaire_annuel : 0..250000 [CHF];
      MANDATORY CONSTRAINT
      !! Si l'employé n'est pas retraité,
      !! son salaire annuel doit être défini
      retraite ==#False AND DEFINED(salaire_annuel);
    END employe;

    ASSOCIATION =
      responsable -- {1} employe;
      role_arbre -- {0..*} arbre;
    END;

  END cadastre_des_arbres;

END Confederation.
```




Chap. 5
Chap. 5

```
INTERLIS 2.3;

MODEL Berne (fr) AT "http://www.interlis.ch/models"
  VERSION "2005-06-16" =

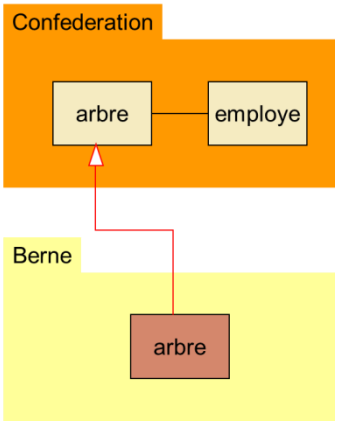
  IMPORTS Confederation;

  TOPIC cadastre_des_arbres EXTENDS Confederation.cadastre_des_arbres =

    CLASS arbre (EXTENDED) =
      remarques: TEXT*2000;
    END arbre;

  END cadastre_des_arbres;

END Berne.
```



```
INTERLIS 2.3;

CONTRACTED SYMBOLOGY MODEL AbstractSymbology (en)
 AT "http://www.interlis.ch/models"
 VERSION "2005-06-16" =

 DOMAIN
  Koord2D (ABSTRACT) = COORD NUMERIC, NUMERIC;

 TOPIC SignsTopic =

  CLASS Symbol EXTENDS INTERLIS.SIGN =
   PARAMETER
    Pos: MANDATORY Koord2D;
  END Symbol;

 END SignsTopic;

END SimpleSignsSymbology.

/** Description de la présentation
*/
MODEL graphique_simple =
 CONTRACT ISSUED BY Unknown;
 IMPORTS SimpleSignsSymbology;
 IMPORTS Confederation;

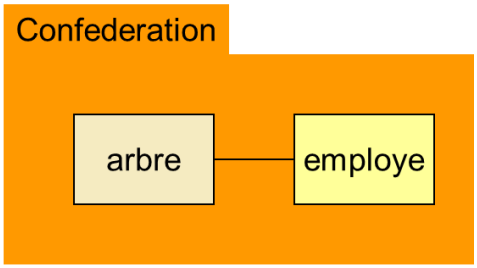
 SIGN BASKET SimpleSignsBasket ~ SimpleSignsSymbology.SignsTopic;

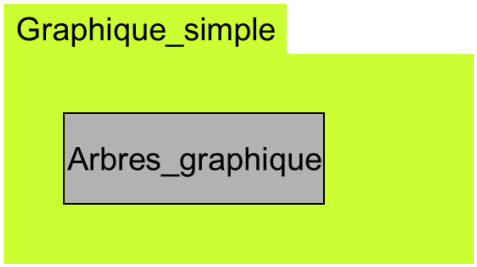
 TOPIC arbres_description_de_presentation =
  DEPENDS ON Confederation.cadastre_des_arbres;

  GRAPHIC arbres_graphique BASED ON Confederation.cadastre_des_arbres.arbre =
   regle_dessin_symbole OF SimpleSignsSymbology.SignsTopic.Symbol: (
    Sign := {SimpleSignsBasket.arbre};
    Pos := position);
  END arbres_graphique;

 END arbres_description_de_presentation;

END graphique_simple.
```




## 1.3. Aperçu de la structure complète

```
INTERLIS 2.3;
MODEL MensurationCadastrale(fr) AT "http://models.interlis.ch"
  VERSION "2005-06-16" =
  UNIT
    CHF EXTENDS INTERLIS.MONEY;
  DOMAIN
    CoordP = ...;
  TOPIC PointsFixes =
    CLASS PointFixe2D =
      Pos : CoordP;
    END PointFixe2D;
  END PointsFixes;
END MensurationCadastrale.
```

## 1.4. Utiliser des définitions d'un autre modèle

Les définitions contenues dans un modèle peuvent être utilisées dans un autre modèle.

Par exemple, les définitions d'unités contenues dans le modèle « Units » (modèle défini dans l'Annexe F du manuel de référence INTERLIS 2).

Extrait du modèle « Units » :

```
INTERLIS 2.3;
CONTRACTED TYPE MODEL Units (en) AT "http://models.interlis.ch/"
  VERSION "2005-06-06" =

  UNIT
    !! abstract Units
    Area (ABSTRACT) = (INTERLIS.LENGTH*INTERLIS.LENGTH);
    …
    Gon = 200 / PI [INTERLIS.rad];
    …
END Units.
```

Extrait du modèle « MensurationCadastrale » reprenant une définition du modèle « Units » :

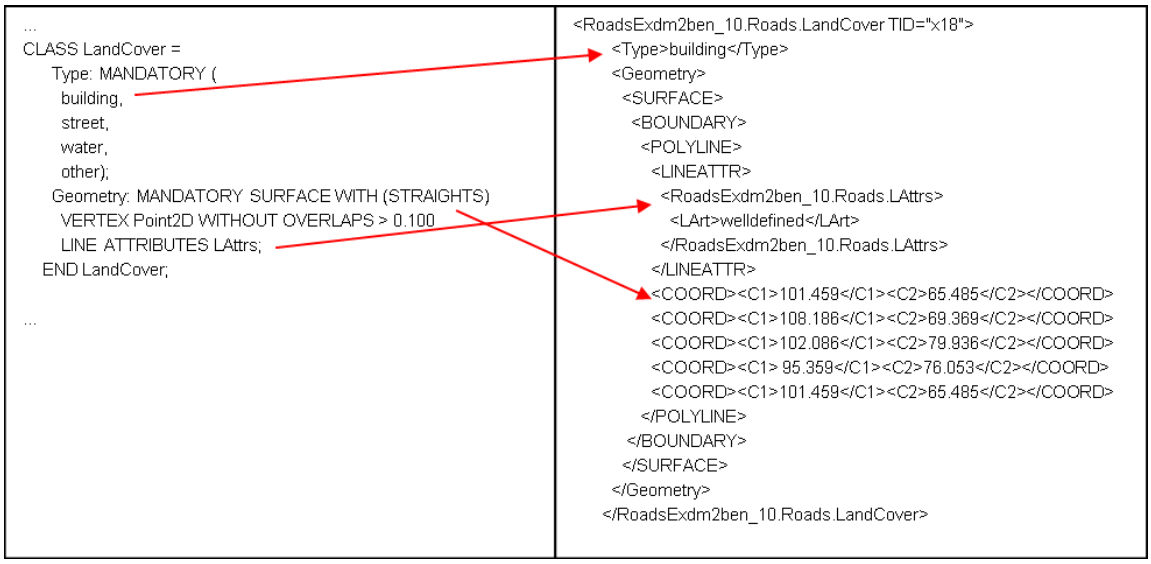
```
INTERLIS 2.3;
MODEL MensurationCadastrale(fr) AT "http://models.interlis.ch/"
VERSION "2005-06-16" =
  IMPORTS Units;
    DOMAIN
      Angle : 0..399 [Units.Gon];
      ...
END MensurationCadastrale.
```





## 1.5. Format d'échange

Le format du fichier de transfert est le format XML (eXtensible Markup Language). Ci-dessous, extrait d'un modèle de données (à gauche) et extrait d'un fichier de transfert correspondant (à droite).




## 1.6. Dépendances entre conteneurs

Les dépendances entre conteneurs (ensembles d'objets qui appartiennent à un même thème) doivent être déclarées explicitement dans la définition du thème.

Règle de syntaxe:

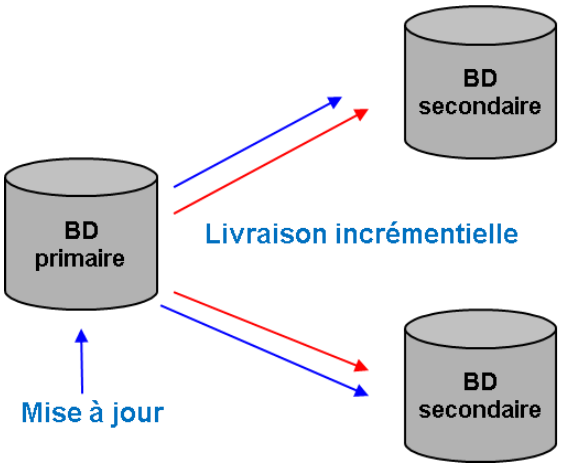
```
TopicDef = 'TOPIC' Topic-Name '='
           { 'DEPENDS' 'ON' TopicRef ';' }
              Definitions
           'END' Topic-Name ';'.
```

Exemple de déclaration d'une dépendance entre conteneurs :

```
TOPIC Habitant =
  ...
END Habitant;
TOPIC Registre_foncier =
  DEPENDS ON Habitant;
  ...
END Registre_foncier;
```

## 1.7. Livraison incrémentielle des données

Après un transfert initial complet d'un ou de plusieurs conteneurs de données, INTERLIS 2 permet de transférer d'une base de données primaire à des bases secondaires uniquement les mises à jour des données.




Genres de transfert

1 - INITIAL - FULL

2 - UPDATE ou DELETE



*© CF-geo, Mattei Alain 09.2019*

# 2. Unités et systèmes de références

## 2.1. Unités de base

Les unités de base sont les unités les plus utilisées qui n'ont pas besoin d'explication, comme par exemple l'unité du mètre [m] ou de la seconde [s].

Les unités de base peuvent être définies de manière abstraite (avec le mot clé « ABSTRACT ») comme par exemple la température.

L'annexe F du manuel de référence INTERLIS 2 définit les unités de base les plus utilisées.

Syntaxe :

```
UnitDef = 'UNIT'
            { Unit-Name
                [ '(' 'ABSTRACT' ')' | '[' UnitShort-Name ']' ]
                [ 'EXTENDS' Abstract-UnitRef ]
                [ '=' ( DerivedUnit | ComposedUnit ) ] ';' }.
DerivedUnit = [ DecConst { ( '*' | '/' ) DecConst }
              | 'FUNCTION' Explanation ] '[' UnitRef ']'.
ComposedUnit = '(' UnitRef { ( '*' | '/' ) UnitRef } ')'.
UnitRef = [ Model-Name '.' [ Topic-Name '.' ] ] UnitShort-Name.
```

Exemple :

```
UNIT
      LENGTH (ABSTRACT);
      ...
      METRE [m] EXTENDS LENGTH;
      ...
```

## 2.2. Unités dérivées

Les unités dérivées se déduisent d'autres unités (multiplication ou division par une constante, fonction), comme par exemple le kilomètre.

Syntaxe :

```
UnitDef = 'UNIT'
            { Unit-Name
                [ '(' 'ABSTRACT' ')' | '[' UnitShort-Name ']' ]
                [ 'EXTENDS' Abstract-UnitRef ]
                [ '=' ( DerivedUnit | ComposedUnit ) ] ';' }.
DerivedUnit = [ DecConst { ( '*' | '/' ) DecConst }
              | 'FUNCTION' Explanation ] '[' UnitRef ']'.
ComposedUnit = '(' UnitRef { ( '*' | '/' ) UnitRef } ')'.
UnitRef = [ Model-Name '.' [ Topic-Name '.' ] ] UnitShort-Name.
```

Exemple :

```
UNIT
      KILOMETRE [km] = 1000 [m];
```

## 2.3. Unités composées

Les unités composées s'obtiennent par la multiplication ou la division d'autres unités (unités de base, dérivées ou composées), comme par exemple l'unité du kilomètre par heure.

Les unités composées peuvent aussi être définies des unités abstraites (par exemple vitesse).

Syntaxe :

```
UnitDef = 'UNIT'
            { Unit-Name
                [ '(' 'ABSTRACT' ')' | '[' UnitShort-Name ']' ]
                [ 'EXTENDS' Abstract-UnitRef ]
                [ '=' ( DerivedUnit | ComposedUnit ) ] ';' }.
DerivedUnit = [ DecConst { ( '*' | '/' ) DecConst }
              | 'FUNCTION' Explanation ] '[' UnitRef ']'.
ComposedUnit = '(' UnitRef { ( '*' | '/' ) UnitRef } ')'.
UnitRef = [ Model-Name '.' [ Topic-Name '.' ] ] UnitShort-Name.
```

Exemple :

```
UNIT
      Newton [N] = (kg*m/s/s);
```

## 2.4. Unités structurées

Les unités structurées (par exemple la date) se composent de plusieurs autres unités, concrètes ou abstraites, devant s'appuyer directement ou indirectement (unités dérivées) sur une unité de base commune (par exemple le temps).

## 2.5. Unités prédéfinies par INTERLIS

Les unités suivantes sont prédéfinies par INTERLIS :

Unités abstraites :

ANYUNIT DIMENSIONLESS LENGTH MASS TIME ELECTRIC_CURRENT TEMPERATURE AMOUNT_OF_MATTER ANGLE SOLID_ANGLE LUMINOUS_INTENSITY MONEY

Unités concrètes :

METER [m], KILOGRAM [kg], SECOND [s], AMPERE [A], DEGREE_KELVIN [K], MOLE [mol], RADIAN [rad], STERADIAN [sr], CANDELA [cd]

Exemple d'une unité composée à partir d'une unité de base concrète INTERLIS :

```
Longueur : 1 .. 10 [INTERLIS.m];
```

## 2.6. Utilisation des définitions d'unités d'un autre modèle

Syntaxe :

```
UnitDef = 'UNIT'
            { Unit-Name
                [ '(' 'ABSTRACT' ')' | '[' UnitShort-Name ']' ]
                [ 'EXTENDS' Abstract-UnitRef ]
                [ '=' ( DerivedUnit | ComposedUnit ) ] ';' }.
DerivedUnit = [ DecConst { ( '*' | '/' ) DecConst }
              | 'FUNCTION' Explanation ] '[' UnitRef ']'.
ComposedUnit = '(' UnitRef { ( '*' | '/' ) UnitRef } ')'.
UnitRef = [ Model-Name '.' [ Topic-Name '.' ] ] UnitShort-Name.
```

Exemple d'utilisation des définitions du modèle « Units » de l'Annexe F du manuel de référence INTERLIS 2 :

```
INTERLIS 2.3;
MODEL MensurationCadastrale(fr) AT "http://www.interlis.ch/models"
    VERSION "2005-06-16" =
    IMPORTS Units;
      DOMAIN
        Angle : 0..399 [Units.Gon];
        ...
  END MensurationCadastrale.
```

## 2.7. Coordonnées

Les coordonnées peuvent être définies en 1, 2 ou 3 dimensions

Syntaxe :

```
CoordinateType = 'COORD' NumericType
                   [ ',' NumericType [ ',' NumericType ]
                       [ ',' RotationDef ] ].
RotationDef = 'ROTATION' NullAxis-PosNumber '->' PiHalfAxis-PosNumber.
```

Exemple : coordonnées simples 2 dimensions

```
DOMAIN
       ExCoord = COORD 480000.00 .. 850000.00,
                        60000.00 .. 320000.00;
```

## 2.8. Coordonnées et systèmes de référence

Syntaxe :

```
NumericType = ( Min-Dec '..' Max-Dec | 'NUMERIC' ) [ 'CIRCULAR' ]
              [ '[' UnitRef ']' ]
              [ 'CLOCKWISE' | 'COUNTERCLOCKWISE' | RefSys ].
RefSys = ( '{' [ '[' Axis-PosNumber ']' ] '}'
         | '<' Coord-DomainRef [ '[' Axis-PosNumber ']' ] '>' ).
DecConst = ( Dec | 'PI' | 'LNBASE' ).
NumericConst = DecConst [ '[' UnitRef ']' ].
```

Exemple : coordonnées simples avec unité et système de référence

```
DOMAIN
   ExCoord = COORD
                480000.00..850000.00[m] {CHLV03[1]},
                 60000.00..320000.00[m] {CHLV03[2]};
```

## 2.9. Coordonnées et indication de la rotation

Syntaxe :

```
CoordinateType = 'COORD' NumericType
                   [ ',' NumericType [ ',' NumericType ]
                       [ ',' RotationDef ] ].
RotationDef = 'ROTATION' NullAxis-PosNumber '->' PiHalfAxis-PosNumber.
```

Exemple: coordonnées avec unité, système de référence et rotation

```
DOMAIN
   ExCoord = COORD
                480000.00..850000.00[m] {CHLV03[1]},
                 60000.00..320000.00[m] {CHLV03[2]},
                ROTATION 2 -> 1;
```

# 3. Relations, rôle, intensité, cardinalité

## 3.1. Définitions

Glossaire (Manuel de référence INTERLIS 2, Annexe K)

### 3.1.1 Relation :

Ensemble de paires d'objets (resp. dans le cas général de n-tuples d'objets) que l'on appelle aussi les „objets de relation".

Ensemble de paires d'objets (ou dans le cas général de n-tuples d'objets), qui s'appellent aussi des « objets relationnels ».

Le premier objet de chaque paire appartient à une classe A, le deuxième à une classe B.

L'attribution d'objets aux paires doit être modélisée. On fait la distinction entre :

- les relations vraies (ou réelles) : association, agrégation, composition,
- les relations d'héritage, et
- les attributs de référence (cf. INTERLIS 1)

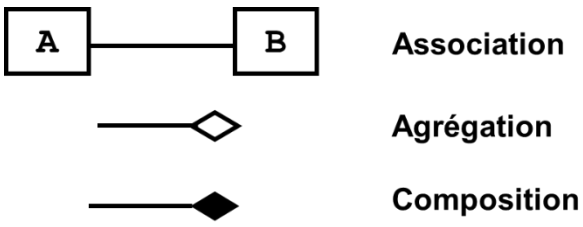
### 3.1.2 Rôle :

Le rôle est l'importance des objets d'une classe dans la relation

- Relation vraie : Rôle = Nom de rôle + Intensité + Cardinalité
- Relation d'héritage : Rôle donné implicitement
- Attribut de référence *: Rôle de la classe de cardinalité 1*

### 3.1.3 Représentation graphique

Dans un diagramme UML les différentes relations vraies sont représentées comme suit :

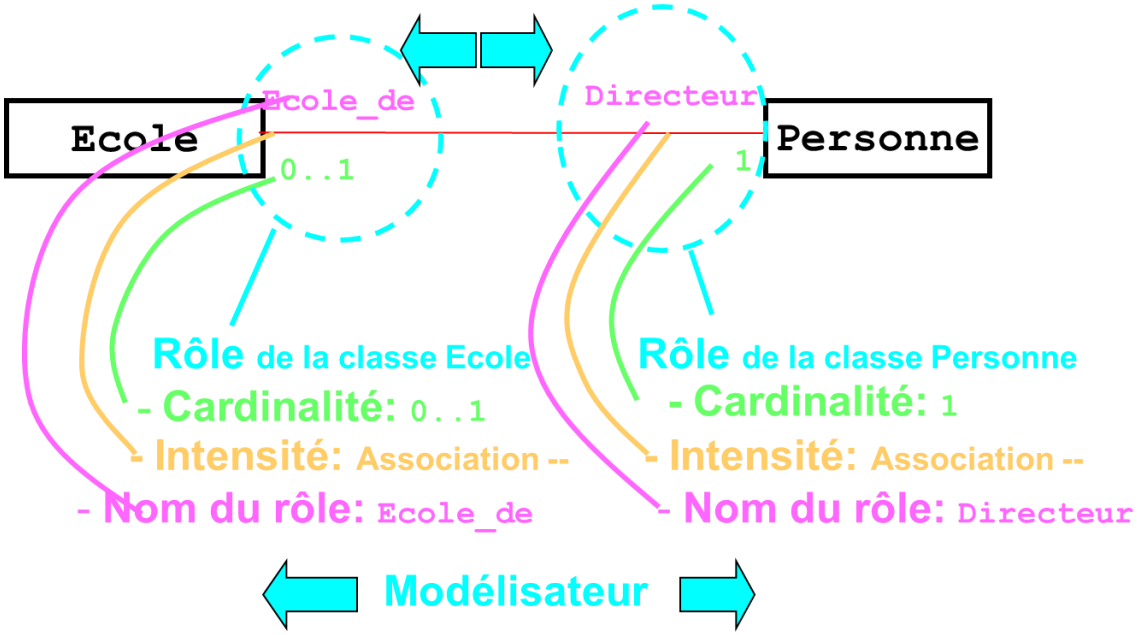




*© CF-geo, Mattei Alain 09.2019*

## 3.2. Syntaxe INTERLIS 2

### 3.2.1 Aperçu




### 3.2.2 Association en général

```
AssociationDef = 'ASSOCIATION' [ Association-Name ]
                   Properties<ABSTRACT,EXTENDED,FINAL,OID>
                   [ 'EXTENDS' AssociationRef ]
                   [ 'DERIVED' 'FROM' RenamedViewableRef ] '='
                   [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
                   { RoleDef }
                   [ 'ATTRIBUTE' ] { AttributeDef }
                   [ 'CARDINALITY' '=' Cardinality ';' ]
                   { ConstraintDef }
                 'END' [ Association-Name ] ';'.
AssociationRef = [ Model-Name '.' [ Topic-Name '.' ] ] Association-Name.
RoleDef = Role-Name Properties<ABSTRACT,EXTENDED,FINAL,HIDING,
                               ORDERED,EXTERNAL>
            ( '--' | '-<>' | '-<#>' ) [ Cardinality ]
            RestrictedClassOrAssRef { 'OR' RestrictedClassOrAssRef }
            [ ':=' Role-Factor ] ';'.
Cardinality = '{' ( '*' | PosNumber [ '..' ( PosNumber | '*' ) ] ) '}'.
```

### 3.2.3 Association simple

```
AssociationDef = 'ASSOCIATION' '='
                     { RoleDef }
                 'END' ';'.
RoleDef = Role-Name '--' ClassRef ';'.
```

Exemple :

```
CLASS Ecole ... ;
CLASS Personne ... ;
ASSOCIATION =
      Ecole_de -- Ecole;
      Directeur -- Personne;
END;
```

### 3.2.4 Cardinalité

La cardinalité définit les nombres minimum et maximum d'objets de chaque classe autorisés dans la relation. Le signe « * » indique qu'il n'y a pas de limite supérieure. La cardinalité indiquée {*} est équivalent à {0..*}.

```
RoleDef = Role-Name [ Cardinality ] ClassRef ';'.
Cardinality = '{' ( '*'
  | PosNumber [ '..' ( PosNumber | '*' ) ] ) '}'.
```

Exemple :

```
ASSOCIATION =
  Ecole_de -- {0..*} Ecole;
  Directeur -- {1} Personne;
END;
```

### 3.2.5 Intensité de la relation

L'intensité d'une relation a une influence directe sur la copie ou la suppression d'objets.

Association (--) : relation entre objets indépendants :

- exemple : personne et adresse
- la copie d'un objet sera liée aux mêmes objets que l'original
- la suppression d'un objet lié aura pour conséquence de supprimer la relation sans supprimer l'objet lié.

Agrégation (-<>) : relation entre des parties et un tout ; une partie peut être partie de plusieurs tout.

- exemple : personne et société
- la copie du tout entraîne la copie de toutes les parties
- la suppression d'un objet lié supprimera la relation sans supprimer le / les objets liés

Composition (-<#>) : relation entre des parties et un tout ; une partie ne peut être que partie d'un seul tout. La relation est de forte intensité.

- exemple : appartement et immeuble
- la copie du tout entraîne la copie de toutes les parties
- la suppression du tout entraîne la suppression de toutes les parties liées

```
RoleDef = Role-Name Properties<ABSTRACT,EXTENDED,FINAL,HIDING,
                               ORDERED,EXTERNAL>
            ( '--' | '-<>' | '-<#>' ) [ Cardinality ]
            RestrictedClassOrAssRef { 'OR' RestrictedClassOrAssRef }
            [ ':=' Role-Factor ] ';'.
```

Exemple :

```
ASSOCIATION =
 role_auto -<#> {1} Auto;
 role_chassis -- {1} Chassis;
END;
```

### 3.2.6 Relations ordonnées

Pour que la relation se fasse dans un ordre particulier, la propriété « ORDEERED » doit être définie.

```
RoleDef = Role-Name Properties<ABSTRACT,EXTENDED,FINAL,HIDING,
                               ORDERED,EXTERNAL>
            ( '--' | '-<>' | '-<#>' ) [ Cardinality ]
            RestrictedClassOrAssRef { 'OR' RestrictedClassOrAssRef }
            [ ':=' Role-Factor ] ';'.
```

Exemple:

```
ASSOCIATION =
 ligneR -- {1} Ligne;
 pointsAppui (ORDERED) -- {2..*} Point;
END;
```

# 4. Condition de cohérence, expression logique, fonction

## 4.1. Exemple : INTERLIS1 → INTERLIS 2

INTERLIS 1

```
TOPIC MunicipaliteTop =
    .....
 TABLE Mise_a_jour_municipalite =
   Validite: (provisoire,valable);
 END Mise_a_jour_municipalite;

 TABLE Municipalite =
   Origine: OPTIONAL-> Mise_a_jour_municipalite
    //Validite = valable//;
    !!c-mc
   Nom: Text*30;
   NoOFS: [1 .. 9999];
   Geometrie: SURFACE WITH (STRAIGHTS,ARCS) VERTEX
 Lkoord
    BASE //CoordP seulement pts
 PFP,PtLimite,PtlimiteTerr //
    WITHOUT OVERLAPS > 0.050;
   Validite: (valable, litigeux);
   Origine: OPTIONAL TEXT*30;
  IDENT Nom,NoOFS;
 END Municipalite;

END MunicipaliteTop.
```

INTERLIS 2

```
TOPIC MunicipaliteTop DEPENDS ON PointsFixes,
 Lime_terr  =
     .....
  CLASS Mise_a_jour_municipalite =
    Validite: (provisoire,valable);
  END Mise_a_jour_municipalite;

  CLASS Municipalite =
    Nom: MANDATORY Text*30;
    NoOFS: MANDATORY 1 .. 9999;
    Geometrie: MANDATORY SURFACE WITH
 (STRAIGHTS,ARCS)
     VERTEX CoordP WITHOUT OVERLAPS >0.050;
    Validite: MANDATORY (valable, litigeux);
    Origine: TEXT*30;
    UNIQUE Nom, NoOfs;
    EXISTENCE CONSTRAINT Geometrie REQUIRED IN
    PointsFixes.PFP:Geometrie OR
    BiensFonds.PtLimite:Geometrie OR
    Lime_terr.PtLimiteTerr:Geometrie;
  END Municipalite;

  ASSOCIATION Origine_municipalite =
    Origine -- {0..1} Mise_a_jour_municipalite;
    Municipalite_enregistree -- {0..*} Municipalite;
    MANDATORY CONSTRAINT
    Origine -> Validite == #valable;
 END Origine_municipalite;

END MunicipaliteTop.
```

## 4.2. Définitions et aperçu

### 4.2.1 Définitions

Une **condition de cohérence** (appelée également règle d'intégrité) est la définition d'une restriction à laquelle tous les objets doivent obéir.

S'il existe une certaine liaison entre différents attributs d'un objet ou entre différents objets, alors celle-ci est décrite par une condition de cohérence.

Des conditions de cohérence sont formulées pour une

- Classe (→Règle ClassDef)
- Association (→Règle AssociationDef)
- Vue (→Règle ViewDef)

Différents types de restrictions (conditions de cohérence) sont à distinguer :

- La **condition d'unicité** touche ensemble tous les objets d'une classe



- La **condition obligatoire** touche les objets particuliers
- La **condition plausible** touche les objets particuliers ; le pourcentage d'objets respectant la règle sera défini et devra être respecté
- L'**exigence d'existence** touche les objets particuliers ; cette condition permet d'exiger que la valeur d'un attribut de chaque objet de la classe contrainte apparaisse dans un attribut donné d'un objet d'une autre classe

Voir le chapitre 2.12 du Manuel de référence INTERLIS 2.

## 4.3. Détails des conditions de cohérences

### 4.3.1 Condition d'unicité

La condition d'unicité concerne tous les objets d'une classe et décrit, quelle combinaison d'attributs doit être unique (forme une clé).

Cette condition ne peut pas être l'identification des objets.

Règle de syntaxe :

```
UniquenessConstraint = 'UNIQUE' [ 'WHERE' Logical-Expression ':' ]
                          ( GlobalUniqueness | LocalUniqueness ) ';'.
GlobalUniqueness = UniqueEl.
UniqueEl = ObjectOrAttributePath { ',' ObjectOrAttributePath }.
LocalUniqueness = '(' 'LOCAL' ')'
                    StructureAttribute-Name
                      { '->' StructureAttribute-Name } ':'
                              Attribute-Name { ',' Attribute-Name }.
```

Exemple :

```
CLASS Employe =
    NoPersonel: MANDATORY TEXT*30;
    ....
    !! Chaque employé a un numèro
    !! personnel unique
  UNIQUE NoPersonnel;
END Employe;
```

### 4.3.2 Condition obligatoire

La condition obligatoire (ou invariante de classe) concerne un objet particulier. Tous les objets de la classe doivent remplir la condition à chaque moment.

Règle de syntaxe :

```
MandatoryConstraint = 'MANDATORY'
  'CONSTRAINT' Logical-Expression ';'.
```

Exemple :

```
CLASS Employe =
    Prenom: MANDATORY TEXT*30;
    ....
    retraite: BOOLEAN;
    SalaireAnnuel: 0 .. 200000 [EUR];
  MANDATORY CONSTRAINT
    !! si l'employé n'est pas retraité,
    !! un salaire annuel doit être défini
    retraite == #FALSE   AND
    DEFINED (SalaireAnnuel);
END Employe;
```

### 4.3.3 Exigence d'existence

Une exigence d'existence oblige que la valeur d'un attribut de chaque objet de la classe contrainte apparaisse dans un attribut donné d'un objet d'une autre classe

Règle de syntaxe :

```
ExistenceConstraint = 'EXISTENCE' 'CONSTRAINT'
                        AttributePath 'REQUIRED' 'IN'
                          ViewableRef ':' AttributePath
                           { 'OR' ViewableRef ':' AttributePath } ';'.
```

Exemple :

```
CLASS Point =
    Lieu: Coordonnee;
END Point;
CLASS Rue =
    Axe: POLYLINE WITH (STRAIGHTS,ARCS)
     VERTEX (Coordonnee);
  EXISTENCE CONSTRAINT
    !! Tout les points d'appui de l'axe doivent
    !! correspondre à l'attribut «Lieu» d'un
    !! objet de la classe point
    Axe REQUIRED IN Point:Lieu;
END Rue;
```

## 4.4. Fonctions

Voir le chapitre 2.14 du Manuel de référence INTERLIS 2.

Les fonctions standard suivantes sont prédéfinies :

**FUNCTION myClass (Object: ANYSTRUCTURE): STRUCTURE;**

Elle fournit la classe de l'objet.

```
FUNCTION isSubClass (potSubClass: STRUCTURE; potSuperClass:
STRUCTURE): BOOLEAN;
```

Son résultat est vrai (true) si la classe du premier argument coïncide avec la classe ou une sous-classe du deuxième argument.

```
FUNCTION isOfClass (Object: ANYSTRUCTURE; Class: STRUCTURE):
BOOLEAN;
```

Son résultat est vrai (true) si l'objet du premier argument appartient à la classe ou à une sous-classe du deuxième argument.

```
FUNCTION elementCount (bag: BAG OF ANYSTRUCTURE): NUMERIC;
```

Elle fournit le nombre d'éléments que renferme le Bag (ou la liste).

```
FUNCTION objectCount (Objects: OBJECTS OF ANYCLASS): NUMERIC;
```

Elle fournit le nombre d'objets que contient l'ensemble d'objets donné.

```
FUNCTION len (TextVal: TEXT): NUMERIC;
```

```
FUNCTION lenM (TextVal: MTEXT): NUMERIC;
```

Elle fournit la longueur du texte sous la forme du nombre de ses caractères.

```
FUNCTION trim (TextVal: TEXT): TEXT;
```

```
FUNCTION trimM (TextVal: MTEXT): MTEXT;
```

Elle fournit le texte débarrassé de ses espaces initiaux et finaux.

```
FUNCTION isEnumSubVal (SubVal: ENUMTREEVAL; NodeVal: ENUMTREEVAL):
BOOLEAN;
```

Elle retourne la valeur true (vrai) lorsque SubVal est un sous-élément, donc un sous-noeud ou une feuille du noeud NodeVal.

```
FUNCTION inEnumRange (Enum: ENUMVAL;
                      MinVal: ENUMTREEVAL;
                      MaxVal: ENUMTREEVAL): BOOLEAN;
```

Elle retourne la valeur true (vrai) lorsque l'énumération appartient à Enum, qu' elle est ordonnée et qu'elle est comprise entre les limites MinVal et MaxVal.

```
FUNCTION convertUnit (from: NUMERIC): NUMERIC;
```

Elle convertit la valeur numérique du paramètre "from" en valeur numérique de renvoi en tenant compte des unités liées au paramètre. N'est applicable que lorsque les unités dérivent d'une unité commune.

```
FUNCTION areAreas (Objects: OBJECTS OF ANYCLASS;
                   SurfaceBag: ATTRIBUTE OF @ Objects
                               RESTRICTION (BAG OF ANYSTRUCTURE);
                   SurfaceAttr: ATTRIBUTE OF @ SurfaceBag
                                RESTRICTION (SURFACE)): BOOLEAN;
```

Elle vérifie si les surfaces forment une partition du territoire, conformément à l'ensemble d'objets (premier paramètre) et à l'attribut (troisième paramètre).

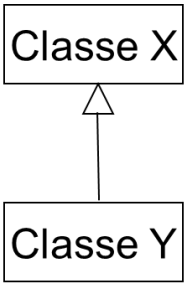
# 5. Héritage pour éléments de base

## 5.1. L'héritage

La notion d'héritage est issue du concept de modélisation orientée objet.

Le rôle premier de l'héritage est de favoriser une économie de représentation et de traitement. La factorisation de ce qui est commun à plusieurs sous-classes dans une superclasse offre des avantages capitaux. Vous pouvez omettre d'écrire dans la définition de toutes les sous-classes ce qu'elles héritent des superclasses.

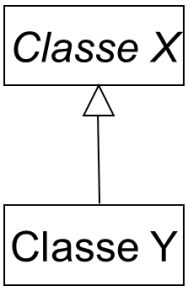
Moins on écrit d'instructions, plus fiable et plus facile à maintenir sera le code.




### 5.1.1 Héritage et classes abstraites

Lors de la factorisation de ce qui est commun à plusieurs sous-classes, il est usuel de définir la superclasse en tant que **classe abstraite**, notée en langage UML en italique.

Une classe abstraite ne contient **aucun objet**.

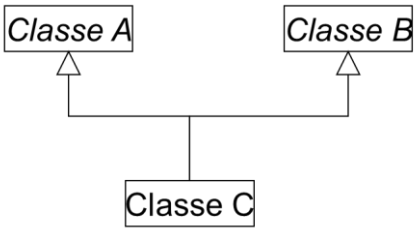



### 5.1.2 Le multihéritage

Une classe hérite de plusieurs classes à la fois.

Exemples :

- Un ordinateur portable est à la fois un ordinateur et un bagage !
- Un artiste de cirque est un clown, un trapéziste, un jongleur, etc.




## 5.2. Le polymorphisme, conséquence de l'héritage !

Le polymorphisme, conséquence directe de l'héritage, permet à un même message, dont l'existence est prévue dans une superclasse, de s'exécuter différemment, selon que l'objet qui le reçoit est d'une sous-classe ou d'une autre.

Exemple :

La souris de votre PC ! Cliquez partout sur l'écran, en fonction de l'endroit où elle se trouve, le résultat du clic ne sera pas le même.

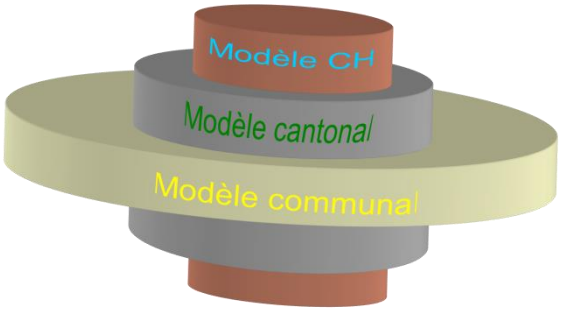
### 5.2.1 Lecture polymorphique avec INTERLIS 2

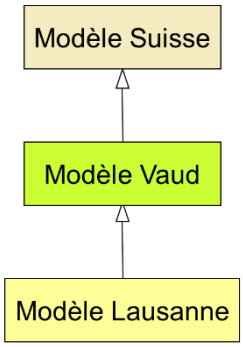
La lecture polymorphique est la capacité pour le système de lire des données issues d'un modèle étendu en ne connaissant que le modèle maître.

Exemple :

Un système ne connaissant que le MD.01-MO-CH sera capable de lire des données issues des modèles cantonaux, par ex : MD.01-MO-FR ou MD.01-MO-VD

### 5.2.2 Compatibilité des modèles, autour d'un noyau commun en utilisant l'héritage




## 5.3. Etendre un thème

```
TopicDef = [ 'VIEW' ] 'TOPIC' Topic-Name
               Properties<ABSTRACT,FINAL>
               [ 'EXTENDS' TopicRef ] '='
                   [ 'BASKET' 'OID' 'AS' OID-DomainRef ';' ]
                   [ 'OID' 'AS' OID-DomainRef ';' ]
                   { 'DEPENDS' 'ON' TopicRef { ',' TopicRef } ';' }
                   Definitions
           'END' Topic-Name ';'.
```

Exemple :

```
TOPIC Confederation =
END Confederation ;

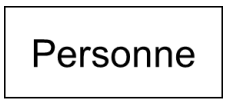
TOPIC Canton EXTENDS Confederation =
END Canton ;
```

### 5.3.1 Classe simple

```
ClassDef = 'CLASS' Class-Name
             Properties<ABSTRACT,EXTENDED,FINAL>
               [ 'EXTENDS' ClassOrStructureRef ] '='
             [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
             ClassOrStructureDef
           'END' Class-Name ';'.
```

Exemple :

```
CLASS Personne =
      Prenom : TEXT*20;
      Nom : TEXT*40;
END Personne;
```

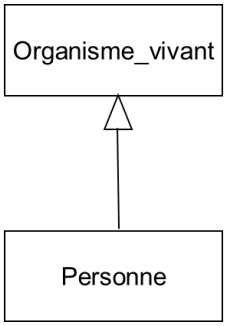



### 5.3.2 Spécialisation d'une classe et changer son nom

```
ClassDef = 'CLASS' Class-Name
             Properties<ABSTRACT,EXTENDED,FINAL>
               [ 'EXTENDS' ClassOrStructureRef ] '='
             [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
             ClassOrStructureDef
           'END' Class-Name ';'.
```

Exemple :

```
CLASS Organisme_vivant =
END Organisme_vivant;
CLASS Personne EXTENDS Organisme_vivant =
END Personne;
```

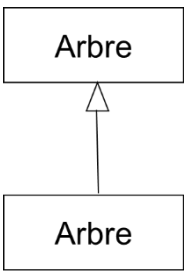



### 5.3.3 Spécialisation d'une classe sans changer son nom

```
ClassDef = 'CLASS' Class-Name
             Properties<ABSTRACT,EXTENDED,FINAL>
               [ 'EXTENDS' ClassOrStructureRef ] '='
             [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
             ClassOrStructureDef
           'END' Class-Name ';'.
```

Exemple :

```
TOPIC Confederation =
  CLASS Arbre =
  END Arbre;
END Confederation ;
TOPIC Canton_VD EXTENDS Confederation =
  CLASS Arbre (EXTENDED) =
  END Arbre;
END Canton_VD;
```



### 5.3.4 Forcer l'héritage

```
ClassDef = 'CLASS' Class-Name
             Properties<ABSTRACT,EXTENDED,FINAL>
               [ 'EXTENDS' ClassOrStructureRef ] '='
             [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
             ClassOrStructureDef
           'END' Class-Name ';'.
```

Exemple :

```
CLASS Organisme_vivant (ABSTRACT) =
END Organisme_vivant;

CLASS Personne EXTENDS Organisme_vivant =
END Personne;
```

### 5.3.5 Empêcher l'héritage

```
ClassDef = 'CLASS' Class-Name
             Properties<ABSTRACT,EXTENDED,FINAL>
               [ 'EXTENDS' ClassOrStructureRef ] '='
             [ ( 'OID' 'AS' OID-DomainRef | 'NO' 'OID' ) ';' ]
             ClassOrStructureDef
           'END' Class-Name ';'.
```

Exemple :

```
CLASS Arbre (FINAL) =
END Arbre;

CLASS Feuillu EXTENDS Arbre = !! incorrect,
END Feuillu;                  !! parce que la classe Arbre
                              !! est définie comme "FINAL".
```

### 5.3.6 Attribut avec un domaine défini incomplet

```
AttributeDef = [ [ 'CONTINUOUS' ] 'SUBDIVISION' ]
                 Attribute-Name
Properties<ABSTRACT,EXTENDED,FINAL,TRANSIENT>
                   ':' AttrTypeDef
                       [ ':=' Factor { ',' Factor } ] ';'.
```

Exemple :

```
Nom (ABSTRACT) : TEXT;  !! longueur maximale du texte manque!
Longueur (ABSTRACT) : NUMERIC; !! intervalle manque!
...
Nom (EXTENDED) : TEXT*200;
Longueur (EXTENDED) : 0 .. 100 [m];
```

### 5.3.7 Attribut avec un domaine restreint

```
AttributeDef = [ [ 'CONTINUOUS' ] 'SUBDIVISION' ]
               Attribute-Name
Properties<ABSTRACT,EXTENDED,FINAL,TRANSIENT>
                 ':' AttrTypeDef
                     [ ':=' Factor { ',' Factor } ] ';'.
```

Exemple :

```
CLASS Attraction =
  Nom : TEXT*120;
END Attraction;

CLASS Musee EXTENDS Attraction =
  Nom (EXTENDED) : TEXT*80;
END Musee;
```

### 5.3.8 Attribut avec un intervalle qui ne pourra plus être limité

```
AttributeDef = [ [ 'CONTINUOUS' ] 'SUBDIVISION' ]
               Attribute-Name
Properties<ABSTRACT,EXTENDED,FINAL,TRANSIENT>
                 ':' AttrTypeDef
                     [ ':=' Factor { ',' Factor } ] ';'.
```

Exemple :

```
CLASS Attraction =
  Nom (FINAL) : TEXT*120;
END Attraction;

CLASS Musee EXTENDS Attraction =
  Nom (EXTENDED) : TEXT*80;   !! incorrect
END Musee;
```

### 5.3.9 Spécialisation de la définition du type d'attribut (domaine)

```
DomainDef = 'DOMAIN'
              { Domain-Name Properties<ABSTRACT,FINAL>
                  [ 'EXTENDS' DomainRef ] '='
              ( 'MANDATORY' [ Type ] | Type ) ';' }.
```

Exemple :

```
DOMAIN
  Normal = 0.00 .. 7.99;
  Precis EXTENDS Normal = 0.0000 .. 7.9949;
!!  correct, les valeurs de „Precis" peuvent aussi être
!!  représentées comme valeurs de „Normal".
```

```
  Precis EXTENDS Normal = 0.0000 .. 7.9999;
!!  faux, la valeur maximale arrondie (à 2 chiffres
!!  après le point virgule) est à l'extérieur de
!!  domaine de „Normal“ (plus grand que le maximum).
```

### 5.3.10 Spécialisation d'une énumération (en ajoutant de nouvelles valeurs)

Exemple :

```
Couleur = (bleu,
           vert,
           rouge);

CouleurPrecise EXTENDS Couleur =
          (rouge (orange, rose, bordeaux));
```

### 5.3.11 Spécialisation d'une énumération (en ajoutant une contrainte)

Exemple :

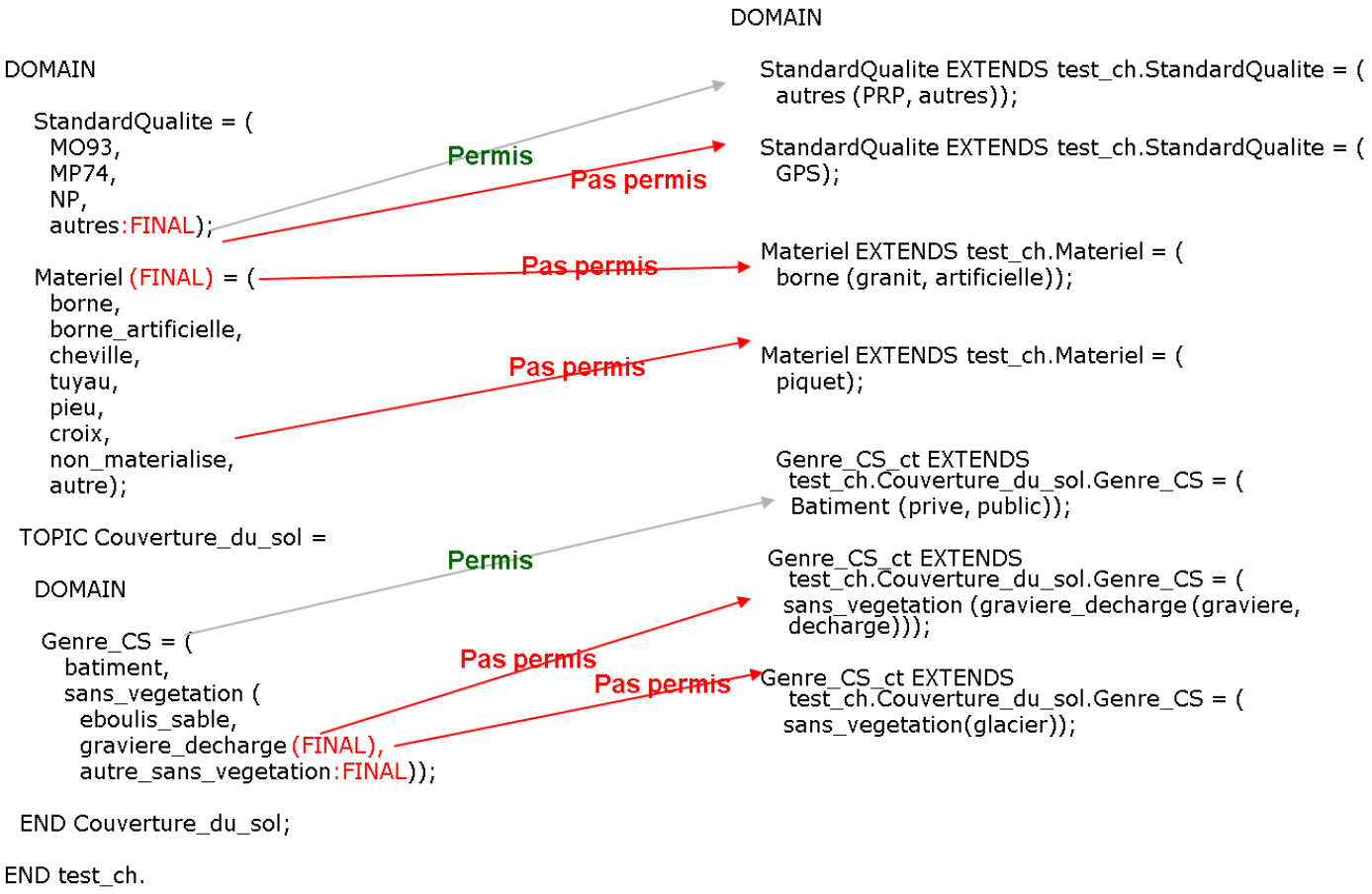
```
Couleur = (bleu,
           vert,
           rouge);

CouleurPlus EXTENDS Couleur = (noir, blanc);

CouleurRGB = (bleu,
              vert,
              rouge:FINAL);
```

Dans cet exemple, l'attribut FINAL dans la CouleurRGB empêche l'ajout de toute valeur supplémentaire dans une extension de CouleurRGB.

### 5.3.12 Exemple pratique




### 5.3.13 Associations

Les cardinalités peuvent être aussi limitées.

Exemple :

```
CLASS Vehicule ...;

CLASS Roue ...;

ASSOCIATION Roues =
  vehicule -- {1} Vehicule;
  roue -- {0..*} Roue;
END Roues;

CLASS Velo EXTENDS Vehicule ...;

ASSOCIATION RouesVelo EXTENDS Roues =
  roue (EXTENDED) -- {2} Roue;
END RouesVelo;
```

# 6. Identifiants d'objets

Les deux identifiants **standard** utilisés en INTERLIS sont présentés ci-dessous.

**STANDARDOID** = OID TEXT*16 :

- les valeurs d'OID de domaines de valeurs d'OID textuels, composés de 16 caractères, doivent satisfaire aux règles du type d'ID XML,
- le premier caractère doit être une lettre ou un trait de soulignement, suivi ensuite de lettres, de chiffres, de points, de signes moins ou de traits de soulignement,
- l'utilisation de deux points ( : ) est interdite,
- exemple : ch00000000000001.

**UUID** = OID TEXT*36 :

- les valeurs d'UUID de domaines de valeurs d'UUID textuels, composés de 36 caractères, doivent être codées sur 128 bits,
- est considéré « universel unique »,
- est produit en utilisant des composantes pseudo-aléatoires, ainsi que les caractéristiques d'un ordinateur (numéro de disque dur, adresse MAC, etc.),
- se présente sous la forme de groupes de caractères hexadécimaux en minuscules séparés par des tirets,
- doit être pris au sens de « unicité très probable »,
- exemple : 110e8400-e29b-11d4-a716-446655440000.

# 7. Outils INTERLIS 2

## 7.1. Compilateur INTERLIS ili2c

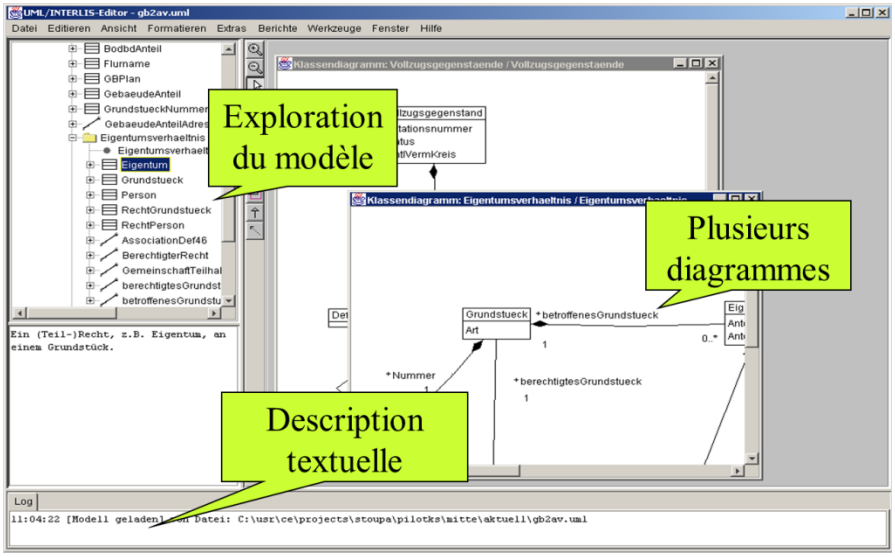
- Contrôle un modèle INTERLIS 1 et 2
- Traduit INTERLIS 1 ↔ INTERLIS 2
- Produit un modèle XML d'un modèle INTERLIS (Description de format INTERLIS 2)
- Produit un fichier FMT d'un modèle INTERLIS (Description de format INTERLIS 1)
- Gratuit (compilateur INTERLIS)

## 7.1. Checker ilivalidator

- Contrôle si les données, disponibles en format de transfert d'INTERLIS 2 (*.xtf), correspondent au modèle correspondant (*.ili).
- Utiliser le model repository fédéral contenant les modèles de base en INTERLIS 2 (https://models.interlis.ch/).
- Manuel en ligne : manuel ilivalidator

## 7.2. Editeur UML/INTERLIS

- Outil permettant de travailler des modèles UML/INTERLIS
- Un modèle de base peut intégrer plusieurs diagrammes
- Spécifique pour INTERLIS
- Lit, examine et décrit les modèles INTERLIS
- Gratuit (UML INTERLIS editor)



## 7.3. Documents et outils pour INTERLIS 2

- Manuel de référence INTERLIS 2.3, COSIG, 2006

  Manuel de référence INTERLIS 2
- Manuel utilisateur INTERLIS 2.3 : La modélisation de données à référence spatiale - Une introduction placée sous le signe des langages UML et INTERLIS (version adaptée et étendue à INTERLIS 2.3), COSIG

  Manuel utilisateur INTERLIS 2
- Glossaire du manuel de référence INTERLIS 2.3, COSIG

  Glossaire du manuel de référence

## 7.4. Outil pour valider des données en INTERLIS 2

- Outil ilivalidator pour valider des données d'un fichier XTF selon un modèle INTERLIS (fichier ILI), projet GitHub de C. Eisenhut.

  outil ilivalidator

## 7.5. Transformation d'un modèle INTERLIS 1 en 2

- A l'aide du compilateur INTERLIS
- Le compilateur peut aussi convertir INTERLIS 2 à 1 -> formuler nouveaux modèles en INTERLIS 2

## 7.6. Transformation des données INTERLIS 1 en 2

- Compacter les données (ficher ITF) et le modèle (ficher ILI) dans un fichier ZIP
- Transférer le fichier ZIP sur le serveur FTP ftp.infogrips.ch
  - Utilisateur : itf2xml
  - Mot de passe : sa propre adresse de courriel
- Le résultat est envoyé par courriel

## 7.7. Outils XML

- Checker: XMLvalidator
- Viewer: XML-Spy

## 7.8. Liens utiles

- https://www.interlis.ch/
- https://www.geo.admin.ch/fr/geoinformation-suisse/geodonnees-de-base/modeles-geodonnees.html
- http://www.eisenhutinformatik.ch/interlis/
- https://www.cadastre.ch/fr/av/method/modell.html
- https://www.safe.com/fme/fme-desktop/

# 8. Bibliographie


- Cours Apprentissage d'INTERLIS 2 par la pratique, EIVD, 2004
- Cours INTERLIS 2, CF-geo, Robert Balanche, octobre 2013
- Cours INTERLIS 2, CF-geo, Isabelle Pointet, octobre 2016
- eCH-0031 INTERLIS 2-Manuel de référence version 2.4
  ➔ https://www.ech.ch/fr/standards/38926
- Manuel utilisateur INTERLIS 2.3 : La modélisation de données à référence spatiale - Une introduction placée sous le signe des langages UML et INTERLIS (version adaptée et étendue à INTERLIS 2.3), COSIG, 2009

  ➔ Manuel utilisateur INTERLIS 2
- Glossaire du manuel de référence INTERLIS 2.3, COSIG

  ➔ Glossaire du manuel de référence

# 9. Exercices – INTERLIS 2

## 9.1. Transcription 1:1 du modèle zone1.ili en INTERLIS 2

Transcrire le modèle de données Plan de zone « zone1_ex_chap1.ili » décrit en INTERLIS 1 en INTERLIS 2 :

- Remplacer la syntaxe INTERLIS 1 par la syntaxe INTERLIS 2
- Ne pas modéliser les relations
- Ne pas modéliser les unités
- Contrôler votre modèle à l'aide du compilateur INTERLIS 2
- Temps à disposition : 50 min.

## 9.2. Modélisation des unités

Modéliser les unités utiles pour le modèle de données « zone2_ex_chap2.ili » décrit en INTERLIS 2 :

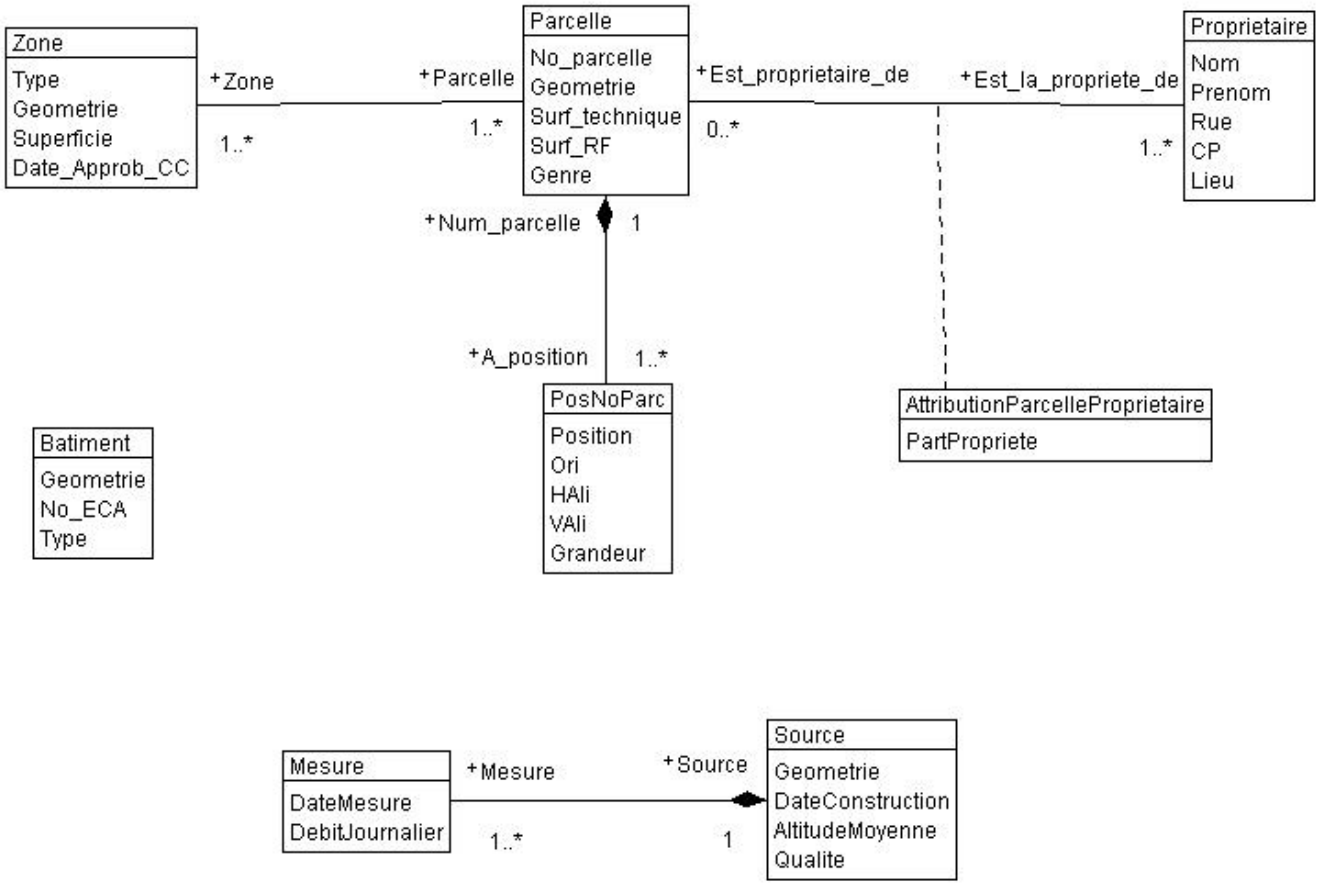
- A l'aide du compilateur, créer le modèle interne prédéfini INTERLIS 2
- Importez le modèle Units.ili et utilisez dans la mesure du possible les unités déjà existantes
- Importez le modèle Time.ili et utilisez dans la mesure du possible les unités déjà existantes
- Modélisez et décrivez les unités utiles du modèle zone2_ex_chap3.ili
- Temps à disposition : 40 min.


*© CF-geo, Mattei Alain 09.2019*

## 9.3. Modélisation des associations

Modéliser les associations utiles pour le modèle de données « zone2_ex_chap3.ili » décrit en INTERLIS 2 :

- Se référer au diagramme UML ci-dessous
- Modélisez et décrivez les associations utiles du modèle zone2_ex_chap3.ili
- Temps à disposition : 40 min.






*© CF-geo, Mattei Alain 09.2019*

## 9.4. Modélisation avec éditeur UML/INTERLIS

Exercice de modélisation de bâtiment et logement avec l'éditeur UML/INTERLIS :

- Déroulement de l'exercice : Exercice_Batiments_Logements.pdf
- Temps à disposition : 50 min.

## 9.5. Exemples d'utilisation de INTERLIS 2

INTERLIS 2 est déjà utilisé pour modéliser et échanger des géodonnées entre professionnels de la géomatique.

En voici deux exemples significatifs :

- Mensuration cadastrale, nouveau modèle DM.Flex :
  ➔ https://www.cadastre.ch/fr/manual-av/method/dm-flex.html

- Normalisation des géodonnées de l'aménagement du territoire NORMAT2 :
  ➔ https://www.vd.ch/themes/territoire-et-construction/amenagement-du-territoire/plan-daffectation-communal/directive-normat/